Panchmeshali Platform — Development Documentation

> ***Version:*** *1.0.0* · ***Date:*** *March 2026* · ***Branch:*** `demo-work`
> ***Repository:*** `iampratikdas/demo_panchmeshali_clusters`

Table of Contents

---

1. Platform Overview

Panchmeshali is a Bengali literary community platform for writers, readers, and publishers. It includes:

Component	Stack	Role
`backend_wen`	Node.js · Express · MongoDB · MySQL	REST API server
`newww`	Vite · React 19 · TypeScript	Admin Portal (web)
Mobile App *(planned)*	Android / iOS	Writer-facing mobile app

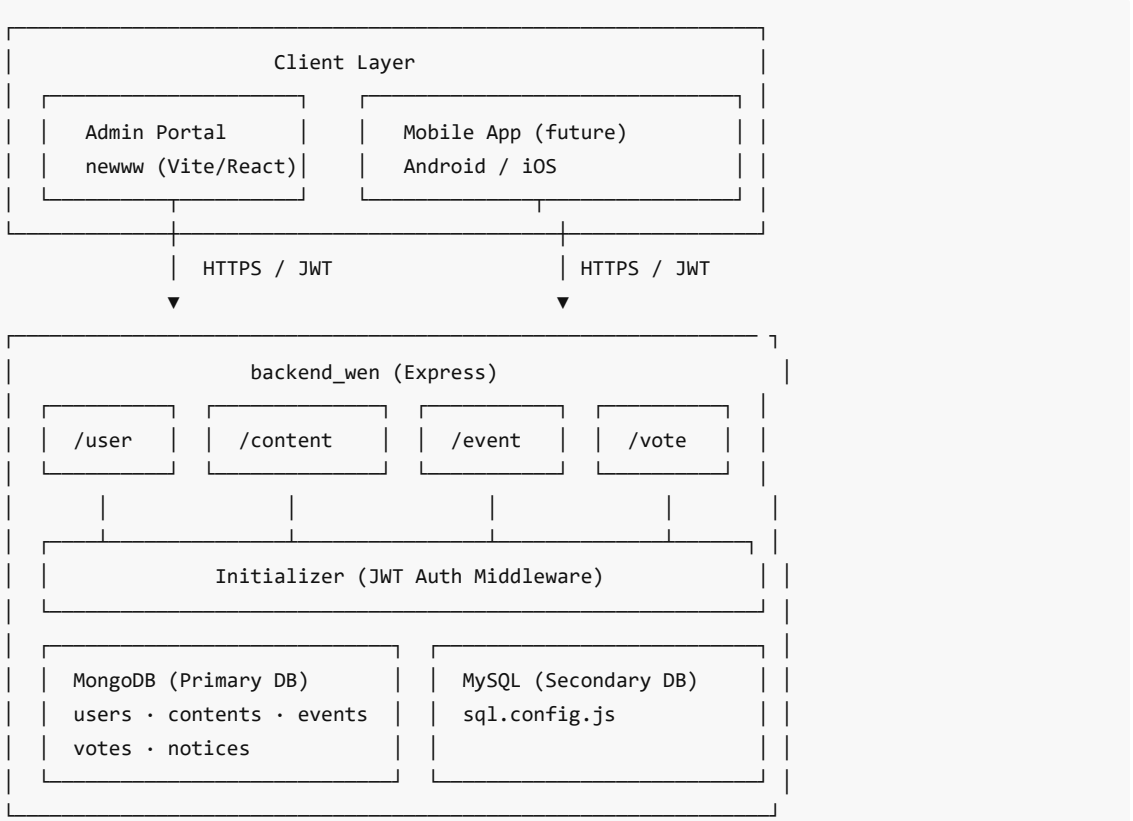
The platform enables:

- Writers to **submit stories, poems, and episodes** for events or general publication
- Admins to **review, mark, and publish** submitted content
- Community **voting** on shortlisted content
- **Event management** — writing competitions, book challenges, episode-wise contests
- **Real-time chat** between admins and writers

- **Certificate** generation for winners and participants

---

2. System Architecture



---

3. Backend — `backend_wen`

3.1 Tech Stack

Library	Version	Purpose
Express	^4.x	HTTP server and routing
Mongoose	^8.x	MongoDB ODM
MySQL2	^3.x	MySQL/MariaDB driver
Passport.js	^0.7	Google OAuth 2.0 strategy
JSON Web Token	^9.x	Stateless authentication
Nodemailer	^6.x	Transactional email
Multer	^1.x	File upload handling
Joi	^17.x	Request body validation
express-rate-limit	^7.x	API rate limiting

Swagger UI Express	^5.x	Auto-generated API docs
bcrypt	^5.x	Password hashing
moment	^2.x	Date/time formatting

3.2 Server Bootstrap

`app.js` contains the `Server` class which bootstraps in 4 phases:

```
Phase 1 — Middlewares
  CORS · rate-limit · express-session · Passport init · body-parser

Phase 2 — Database
  SetupDatabase() → MongoDB connection
  sql.config.js   → MySQL connection pool

Phase 3 — Routes
  /api/v1/* → Route.js → [UserRoutes, ContentRoutes, EventRoutes, VotingRoutes]

Phase 4 — Listen
  app.listen(PORT)
```

3.3 Database Connections

MongoDB (Primary)

- **Connection:** `mongoose.connect()` via `SetupDatabase()` in `database/`
- **Collections:** `users`, `contents`, `events`, `votes`, `noticesadmins`
- **Models:** all defined in `models/monogdb/`

MySQL (Secondary)

- **Connection:** `mysql2.createPool()` in `sql.config.js`
- **Use:** supplementary relational data and analytics queries

3.4 Authentication & Role System

All protected routes pass through `Initializer.js`:

```
Request header: Authorization: Bearer <JWT>
  → jwt.verify(token)
  → DB lookup: users.findOne({ uid })
  → Role check: user.role ∈ requiredRoles[]
  → next() or 401/403
```

Role	Level	Access
`super_admin`	0	Full system access
`admin`	1	Content, events, users
`author`	2	Own content submission only
`reader`	3	Read + vote only

JWT Payload:

```
{ "uid": "user_unique_id", "email": "user@example.com", "role": "admin" }
```

3.5 API Modules

User Module (`/api/v1/`)

Method	Path	Auth	Description
GET	`/auth/google`	—	Initiate Google OAuth flow
GET	`/auth/google/callback`	—	Google OAuth callback
POST	`/login`	—	Email/password login, returns JWT
POST	`/register`	—	New user registration
GET	`/profile`	JWT	Get current user profile
PUT	`/profile`	JWT	Update profile info
POST	`/forgot_password`	—	Send password reset email
POST	`/reset_password`	—	Confirm password reset
GET	`/users`	Admin	List all users
DELETE	`/user/:uid`	Admin	Delete/deactivate user

Content Module (`/api/v1/`)

Method	Path	Auth	Description
POST	`/submit_contents`	JWT	Submit story/poem/episode
GET	`/contents`	JWT	Paginated content list (filter by status, event, type)
GET	`/notices`	JWT	Fetch published notices
GET	`/certificates`	JWT	Fetch winner certificates
PUT	`/add_marks`	Admin	Admin marks/scores a submission
POST	`/create_notice`	Admin	Create and email a notice to writers

Content submission validation (`ContentValidator` middleware) checks:

- `event_id` , `content_title` , `content` , `is_original` are present
- Date: event must be currently active (`st_dt ≤ now ≤ en_dt`)
- Participation: if parent event exists, user must have participated in it
- Duplicate: user cannot submit to the same event twice

Event Module (`/api/v1/`)

Method	Path	Auth	Description

GET	`/events`	Admin	Hierarchical event tree (recursive nesting by `parent`)
GET	`/events_list`	JWT	Flat event list for writers
POST	`/create_event`	Admin	Create new event
PUT	`/update_event/:eid`	Admin	Update event fields
DELETE	`/delete_event/:eid`	Admin	Soft-delete event

Event Schema fields:

Field	Type	Description
`name`	String	Event name
`description`	String	Full description
`active`	Boolean	Is event currently open
`episode_wise`	Boolean	Enables episode-by-episode submission
`for_book`	Boolean	Enables multi-episode book write mode
`categories`	Array	Allowed content categories
`st_dt` / `en_dt`	Timestamp	Start / end dates
`parent`	String	Parent event ID (for hierarchical events)
`w_count`	Number	Max word count per submission
`sh_list`	Number	Shortlist threshold
`type`	String	`"vote"` or `"judge"`

Voting Module (`/api/v1/`)

Method	Path	Auth	Description
GET	`/vote_list`	JWT	Paginated list of contents for voting
GET	`/top_votes`	JWT	Leaderboard of top-voted submissions
POST	`/vote`	JWT	Cast a vote for a content item

Voting rules enforced in `VotingController`:

- One vote per user per content item per event
- Vote is stored with `uid`, `cont_id`, `eid` in the `votes` collection
- `top_votes` uses MongoDB aggregation pipeline to count and rank

3.6 Email System

Transport: Nodemailer with SMTP credentials from `.env`

Used in:

- **Password Reset** — sends reset link to user email
- **Notice Broadcasting** — admin creates a notice via `POST /create_notice`; the controller fetches all registered writer emails and sends the notice as an HTML email using a custom template

3.7 File Uploads

Library: Multer
Usage: Profile picture and content image uploads
Storage: local disk (configurable to cloud storage in production)

3.8 Swagger Docs

Auto-generated API documentation accessible at:

```
GET /api-docs
```

Defined in `swagger.js` using `swagger-ui-express` and `swagger-jsdoc`.

---

4. Admin Portal — `newww`

4.1 Tech Stack

Layer	Tool
Bundler	Vite 7
UI	React 19 + TypeScript 5.9
Routing	TanStack Router
Data Fetching	TanStack Query
Global State	Jotai
Styling	Tailwind CSS 4
Rich Text	Tiptap (`@tiptap/react`, `@tiptap/starter-kit`)
Animations	Framer Motion
Icons	Lucide React
Validation	Zod

4.2 Features Built

Dashboard (`/`)

- Overview statistics cards (total submissions, active events, pending reviews)
- Quick navigation links to all major sections

Content List (`/content`)

- Paginated list of submitted content
- Filter by status: `All`, `Pending`, `Approved`, `Rejected`, `Under Review`
- Each card shows: title, type badge, status badge, author, word count

Content Detail (`/content/$id`)

- Full content view with rendered HTML body
- Comment thread — displays existing comments; admin can add new comments
- AI quality check button (`checkQualityAI`) — mocked, returns score + feedback
- AI proofreader button (`proofreadAI`) — mocked, returns corrections

Chat (`/chats`)

- Admin–writer messaging interface
- Sidebar list of open chats with unread count indicators
- Message thread with timestamps, sent/received alignment

Users (`/users`)

- User list with search
- Create new user / ban user / remove user
- Send email to individual writer via modal

Settings (`/settings`)

- Application configuration panel

4.3 Submission Wizard (`/submit`)

Two independent multi-step wizards on the same page, toggled by tabs:

Content Submission Wizard (4 steps)

Step	Label	Key Fields	Next Gate
0	Destination	Submission type, Publisher	Publisher required if `new`
1	Details	Story Title, Content Type, Category	Title + Type + Category required
2	Write	Content (RichTextEditor), Background Image (16:9), Cover Image (9:16), Destination	—
3	Review	Original authorship checkbox	Must confirm to submit

Event Submission Wizard (3 steps)

Step	Label	Key Fields	Next Gate
0	Event	Event selector	Event must be selected
1	Write	Content (episode or single mode)	—
2	Review	Original authorship checkbox	Must confirm to submit

Episode / Book Mode (controlled by Event flags)

`episode_wise`	`for_book`	Write Step Behaviour
`false`	`false`	Single content title + editor
`true`	`false`	Single editor; adds "Add next episode" to submission type

`true`	`true`	One episode card pre-rendered; "Add episode" button appends more cards with individual remove buttons

Form Validation (`useSubmissionForm` + Zod schema)

- Publisher required when `newSubmission === 'new'` and not an event
- Category required when `newSubmission === 'new'`
- Episode number required when `newSubmission === 'Add next episode'`
- Image uploads validated for aspect ratio (±0.05 tolerance):
  - Background image: **16:9**
  - Cover image: **9:16**

On Successful Submission

1. API call: `POST /submit_contents`
2. Toast notification shown
3. **Auto-save to Workspace:** a new `WorkspaceFile` is prepended to `workspaceFilesAtom` in the root folder, containing the full title, HTML content, category, publisher, event name, and status `'Pending'`

4.4 Workspace (`/workspace`)

Google Drive-inspired file browser for organising submitted content.

Folder Operations

Action	Description
Create Folder	Modal; random colour assigned; added to `workspaceFoldersAtom`
Rename Folder	Modal; updates name + `modifiedAt`
Delete Folder	Blocked if folder contains children or files

File Operations

Action	Available For
Upload	All file types (pdf, doc, docx, txt)
Download	Non-content files only (pdf, docx, txt)
Share	All files — opens email share modal
Delete	All files — confirmation dialog
Preview	Story / Poem files — opens Content Preview Modal

File Card Display

Type	Icon	Colour	Extra Display
`story`	BookOpen	Purple	Badge · Excerpt · "Click to preview" hover
`poem`	BookOpen	Pink	Badge · Excerpt · "Click to preview" hover

`pdf`	FileText	Red	—
`docx/doc`	FileText	Blue	—
`txt`	FileText	Grey	—

Content Preview Modal (`ContentPreviewModal.tsx`)

Mobile layout: Full-screen bottom sheet sliding up from bottom; drag handle at top.
Desktop layout: Centred rounded modal, max 90vh.

View Mode:

- Renders full HTML content body (`.prose` container)
- Metadata grid: Type · Category · Publisher · Author · Event · Submitted · Updated · Size
- Mobile: metadata is a collapsible accordion

Edit Mode:

- Activated by "Edit" button (header on desktop; footer on mobile)
- Body: **RichTextEditor** (Tiptap — Bold, Italic, Heading, Lists, Blockquote, Undo/Redo)
- Editable fields: Title · Author · Category · Publisher · Event Name
- Dropdowns: Type (`story/poem/doc/txt/pdf`) · Status (`Pending/Reviewing/Approved/Rejected`)
- Mobile: separate sticky "Cancel / Save changes" action bar below header
- Save: writes to `workspaceFilesAtom` , recomputes `excerpt`
- Cancel: discards all draft changes

Seed Data

A demo story **"The Lost Kingdom"** is seeded in the workspace root with full content, category (Fantasy), publisher, author, and excerpt pre-filled.

4.5 Event Management (`/events`)

- **List view** — searchable, shows all events in card grid
- **Create event form** — includes:
  - Name, Description, Start date, End date
  - Word count limit, Shortlist count
  - Categories multi-select
  - Toggle: **Event is Active**
  - Toggle: **Episode Wise** (`episode_wise` flag)
  - Toggle: **For Book** (`for_book` flag)
- Submitting the form calls `createEvent()` which adds to mock storage and invalidates the `events` query

---

4.6 User Management (`/users`)

- Paginated user list with search
- **Create User** modal — name, email, role
- **Ban User** action — sets status to `'banned'`
- **Remove User** action — deletes from store
- **Send Email** modal — compose and send to individual writer

---

4.7 Voting System

Voting is managed via the backend. Admin portal surfaces:

- Voting leaderboard from `GET /top_votes`
- Vote status per content item (has the current user voted?)

---

5. Future Roadmap — Android / iOS Integration

The backend API is already REST-based and JWT-authenticated, making it **ready to serve a mobile app** without any core changes.

5.1 Planned Mobile App Features

Feature	Details
Writer Registration & Login	Google OAuth + email/password via existing `/login`, `/register` endpoints
Content Submission	Writers submit stories/poems/episodes from the app directly to `POST /submit_contents`
Event Discovery	Browse active events, see deadlines and categories
Episode-wise Writing	In-app Tiptap-like editor supporting multi-episode book submission
Voting	Community members vote on shortlisted content via `POST /vote`
Notifications	Push notifications (FCM) for submission approvals, notices, new events
Certificate Download	Download participation/winner certificates via `GET /certificates`
Admin Chat	Writers can message admins and receive replies
Profile Management	View + edit profile, upload avatar via `PUT /profile`

5.2 Integration Plan

```
Mobile App
  ↓ HTTPS
backend_wen REST API (existing — no changes needed for core endpoints)
  ↓
MongoDB + MySQL (existing databases)
```

New backend additions required for mobile:

Addition	Description
FCM Push Notifications	`firebase-admin` SDK; store FCM tokens in `users` collection
Refresh Token endpoint	`POST /refresh_token` for long-lived mobile sessions
File upload to cloud	Replace local Multer storage with S3/Cloudinary for user-uploaded images
Rate limiting per device	Extend `express-rate-limit` with device fingerprinting

App version gate	Middleware to reject outdated app versions

5.3 Tech Recommendations for Mobile

Layer	Recommendation
Framework	React Native (Expo) — shares TypeScript types and API layer with the admin portal
Navigation	Expo Router (file-based routing, mirrors TanStack Router pattern)
State	Jotai (same library already used in admin portal)
Data Fetching	TanStack Query (same library already used in admin portal)
Rich Text Input	`@10play/tentap-editor` (React Native Tiptap wrapper)
Push Notifications	Expo Notifications + Firebase Cloud Messaging
Auth Token Storage	`expo-secure-store` for JWT storage on device

---

6. Changelog

March 2026

Date	Change	Files Affected
08 Mar	`ContentPreviewModal` — edit mode with RichTextEditor for all fields	`ContentPreviewModal.tsx`
08 Mar	`ContentPreviewModal` — mobile-responsive bottom sheet layout	`ContentPreviewModal.tsx`
08 Mar	`FileGrid` — story/poem cards open preview instead of download	`FileGrid.tsx`
08 Mar	`useSubmissionForm` — auto-save submission to workspace root	`useSubmissionForm.ts`
08 Mar	`WorkspaceFile` type extended with `fullContent`, `excerpt`, `category`, `publisher`, `author`, `status`, `eventName`	`types/workspace.ts`
08 Mar	Seeded demo story ("The Lost Kingdom") in workspace root	`store/atoms.ts`
08 Mar	`Events.tsx` — added `episode_wise` and `for_book` toggle switches to event creation form	`routes/Events.tsx`
08 Mar	`Submit.tsx` — episode/book mode UI based on event flags	`routes/Submit.tsx`
08 Mar	`Submit.tsx` — Next button disabled until required step fields are filled	`routes/Submit.tsx`

08 Mar	`Submit.tsx` — Publisher required gate for Content → New Submission	`routes/Submit.tsx`
08 Mar	Documentation — `README_Writers.md` created (frontend reference)	`docs/README_Writers.md`
08 Mar	Documentation — `README_Backend.md` created (backend reference)	`docs/README_Backend.md`
08 Mar	Documentation — `DEVELOPMENT.md` created (this file)	`docs/DEVELOPMENT.md`

Document maintained by the Panchmeshali development team. For questions, raise an issue on the repository or contact the lead developer.